

CircularPad1d#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.padding.CircularP>

Description:

Pads the input tensor using circular padding of the input boundary. Tensor values at the beginning of the dimension are used to pad the end, and values at the end are used to pad the beginning. If negative padding is applied then the ends of the tensor get removed. For N-dimensional padding, use `torch.nn.functional.pad()`. padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 2-tuple, uses $(padding_left, padding_right)$. Note that padding size should be less than or equal to the corresponding input dimension. Input: (C, W_{in}) or (N, C, W_{in}) .

Function Signature

```
class torch.nn.modules.padding.CircularPad1d(padding)
[source]#
```

Parameters

Shape:

Code Examples

```
>>> m = nn.CircularPad1d(2)
>>> input = torch.arange(8, dtype=torch.float).reshape(1, 2, 4)
>>> input
tensor([[[0., 1., 2., 3.],
         [4., 5., 6., 7.]]])
>>> m(input)
tensor([[[2., 3., 0., 1., 2., 3., 0., 1.],
         [6., 7., 4., 5., 6., 7., 4., 5.]]])
>>> # using different paddings for different sides
>>> m = nn.CircularPad1d((3, 1))
>>> m(input)
tensor([[[1., 2., 3., 0., 1., 2., 3., 0.],
         [5., 6., 7., 4., 5., 6., 7., 4.]]])
```

Detailed Documentation

Documentation

Pads the input tensor using circular padding of the input boundary.

Tensor values at the beginning of the dimension are used to pad the end, and values at the end are used to pad the beginning. If negative padding is applied then the ends of the tensor get removed.

For N-dimensional padding, use `torch.nn.functional.pad()`.

padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 2-tuple, uses (`padding_left``\text{padding\_left}``padding_left`, `padding_right``\text{padding\_right}``padding_right`) Note that padding size should be less than or equal to the corresponding input dimension.

Input: (C,Win)(C, W_{in})(C,Win) or (N,C,Win)(N, C, W_{in})(N,C,Win).

Output: (C,Wout)(C, W_{out})(C,Wout) or (N,C,Wout)(N, C, W_{out})(N,C,Wout), where

$W_{out} = W_{in} + \text{padding_left} + \text{padding_right}$
 $W_{out} = W_{in} + \text{padding_left} + \text{padding_right}$